Chương 3: Điều khiển tương tranh

Giới thiệu

Điều khiển tương tranh là gì?

- Là tập hợp các kỹ thuật giúp:
  - Các **giao tác (transactions)** truy cập đồng thời vào dữ liệu
  - Không gây ảnh hưởng lẫn nhau
  - Đảm bảo **lịch giao tác khả tuần tự (serializable schedule)**

Giới thiệu

Mục tiêu nhằm tránh các vấn đề thường gặp:

- Mất dữ liệu cập nhật (Lost Update)
- Đọc dữ liệu rác (Dirty Read)
- Không đọc lại được dữ liệu (Non-repeatable Read)
- Bóng ma (Phantom)

Khái niệm cơ bản

Đơn vị dữ liệu (Data Item)

- Cơ sở dữ liệu được xem như tập hợp các **đơn vị dữ liệu**.
- Một đơn vị dữ liệu có thể là:
  - Một **trường dữ liệu**
  - Một **bản ghi (record)**
  - Một **khối dữ liệu trên đĩa**
  - Một **bảng dữ liệu**
  - Toàn bộ **cơ sở dữ liệu**

Khái niệm cơ bản

Đơn vị dữ liệu (Data Item)

- Kích thước đơn vị dữ liệu ảnh hưởng đến:
  - Mức độ cho phép thực hiện đồng thời
  - Hiệu suất hệ thống
  - Chi phí quản lý khóa

Khái niệm cơ bản

Đơn vị dữ liệu (Data Item)

- Kích thước đơn vị dữ liệu ảnh hưởng đến:
- Ví dụ

TH 1: Đơn vị là khối dữ liệu

- Giao tác T khóa bản ghi B
- Vì B thuộc khối X → T phải khóa toàn bộ X
- Giao tác T' muốn truy cập bản ghi C (cùng khối X)
- → T' phải chờ

TH 2: Đơn vị là bản ghi

- T khóa bản ghi B
- T' khóa bản ghi C
- → Hai giao tác thực hiện đồng thời được

- ➔Cần cân bằng giữa việc lựa chọn đơn vị dữ liệu với hiệu suất điều khiển tương tranh

Khái niệm cơ bản

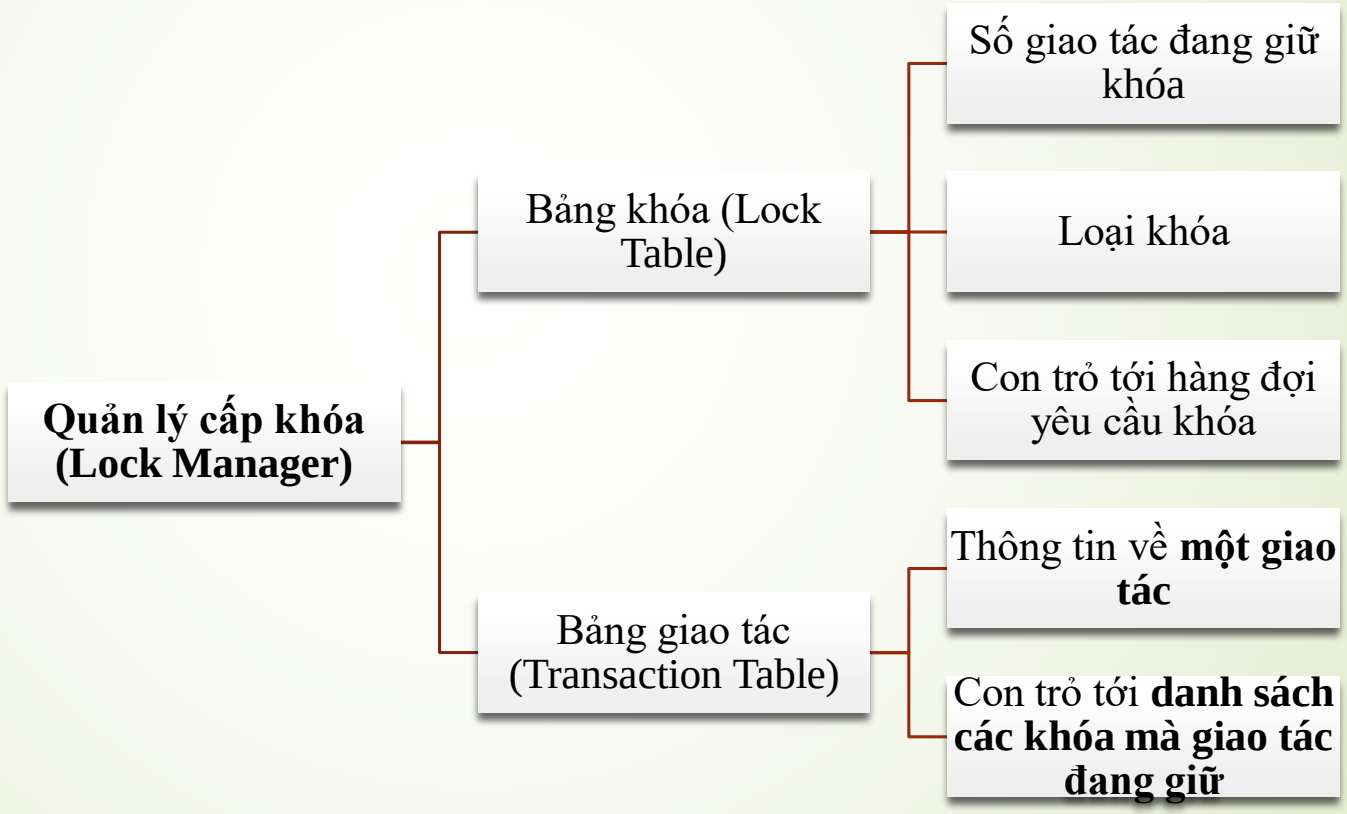
Khóa (Lock)

- Là một biến gắn với đơn vị dữ liệu
- Mô tả trạng thái hành động được phép thực hiện
- **Vai trò của khóa:**
  - Đồng bộ hóa truy cập đồng thời
  - Ngăn xung đột giữa các giao tác
  - Đảm bảo tính nhất quán dữ liệu

Luật cấp khóa

- Hệ quản trị CSDL phải:
  - Theo dõi các **khóa đã được cấp**
  - Quản lý việc **cấp và giải phóng khóa**
- ➔ Chức năng này được gọi là **Quản lý cấp khóa (Lock Manager)**

Luật cấp khóa



Luật cấp khóa

Kiểm tra trước khi cấp khóa

- Trước khi cấp khóa mới, Hệ quản trị CSDL kiểm tra:
  - Giao tác **đã giữ khóa đó chưa**
  - Một giao tác **không được phép giữ cùng một khóa hai lần**.

Dùng khóa để điều khiển tương tranh

- **Các vấn đề khi truy cập đồng thời**
  - Mất dữ liệu cập nhật (Lost Update)
  - Đọc dữ liệu rác (Dirty Read)
  - Không đọc lại được dữ liệu (Non-repeatable Read)
  - Bóng ma (Phantom)
- Các loại khóa được dùng để **ngăn các vấn đề này**.

Mất dữ liệu cập nhật (Lost Update)

Xảy ra khi

- Hai giao tác:
  - Cùng **đọc dữ liệu**
  - Cùng **cập nhật**
  - Một giao tác **ghi đè kết quả giao tác kia**
- ➔Làm mất cập nhật trước đó.

Mất dữ liệu cập nhật (Lost Update)

- Ví dụ: cho 2 giao tác T1, T2 và đơn vị dữ liệu A=50
  - T1: Read(A);A:=A+10;Write(A)
  - T2: Read(A);A:=A+20;Write(A)
- Nếu T1, T2 thực hiện tuần tự thì khi kết thúc, A=80
- Nếu T1, T2 thực hiện đồng thời như hình bên thì khi kết thúc, A=70
- ➔ ***Dữ liệu do T1 ghi đã bị T2 làm mất***

A=50	T1	T2
t1	Read(A)	
t2		Read(A)
t3	A:=A+10	
t4		A:=A+20
t5	Write(A)	
t6		Write(A)

Đọc dữ liệu rác (Dirty Read)

Xảy ra khi

- Một giao tác **đọc dữ liệu chưa được xác nhận** của giao tác khác.

Đọc dữ liệu rác (Dirty Read)

- Ví dụ: cho 2 giao tác T1, T2 và đơn vị dữ liệu A=50
  - Giao tác T1 cập nhật A
  - T1 chưa commit
  - Giao tác T2 đọc A
  - Sau đó T1 bị rollback
- ➔ ***Dữ liệu do T2 đọc không hợp lệ nên gọi là rác***

A=50	T1	T2
t1	Read(A)	
t2	A:=A+10	
t3	Write(A)	
t4		Read(A)
t5		Print(A)
t6	Abort	

Không đọc lại được dữ liệu (Non-repeatable Read)

Xảy ra khi

- Một giao tác đọc **trên cùng dữ liệu hai lần**.
- Giữa hai lần đọc, một giao tác khác **đã thay đổi dữ liệu**
- ➔ giá trị của 2 lần đọc này **khác nhau**

Không đọc lại được dữ liệu (Non-repeatable Read)

- Ví dụ: cho 2 giao tác T1, T2 và đơn vị dữ liệu A=50
  - Nếu T1, T2 thực hiện tuần tự thì các lần đọc A của T2 giống nhau.
  - Nếu T1, T2 thực hiện đồng thời như hình bên thì các lần đọc A của T2 khác nhau.

A=50	T1	T2	
t1	Read(A)		
t2		Read(A)	A=50
t3	A:=A-10		
t4		Print(A)	A=50
t5	Write(A)		
t6		Read(A)	A=40
t7		Print(A)	A=40

Bóng ma (Phantom)

Xảy ra khi

- Một giao tác đang **đọc một tập hợp các đơn vị dữ liệu**.
- Một giao tác khác **thêm hoặc xóa bớt một đơn vị dữ liệu**
- ➔ kết quả **không chính xác**

Bóng ma (Phantom)

- Ví dụ: cho 2 giao tác T1, T2 được xử lý đồng thời
  - A là tập các đơn vị dữ liệu a1, a2, a3,…
  - T1 xử lý trên toàn bộ tập A.
  - Khi T1 đang xử lý, T2 thêm hoặc xóa một hay một số phần tử trong tập A

	T1	T2
t1	Read(A)	
t2	Xử lý 1 trên A	
t3		Thêm ai vào A
t4	Xử lý 2 trên A	
t5		Xoá aj khỏi A
t6	Xử lý 3 trên A	

Livelock (Khóa sống)

- Livelock xảy ra khi:
  - Một giao tác **không được cấp khóa v**ì đơn vị dữ liệu **luôn bị giao tác khác chiếm giữ**
  - ➔ Giao tác **phải chờ rất lâu hoặc có thể không bao giờ được cấp khóa**.

Livelock (Khóa sống)

- Ví dụ: Giả sử nhiều giao tác muốn khóa **X**:
  - **T1** giải phóng khóa X
  - **T2** và **T3** cùng chờ khóa
  - Hệ thống cấp khóa cho **T3 trước T2**
- Sau đó:
  - **T3** giải phóng khóa
  - **T4** lại xin khóa trước **T2**
- Tình trạng này lặp đi lặp lại
- ➔ T2 có thể phải chờ rất lâu hoặc không bao giờ được cấp khóa

Livelock (Khóa sống)

- Giải pháp:
  - Một giải pháp phổ biến: Cấp khóa theo hàng đợi (FIFO)
    - Các giao tác đăng ký xin khóa theo thứ tự
    - Bộ quản lý khóa cấp khóa lần lượt
    - Giao tác đến trước được cấp trước

Deadlock (Khóa chết)

- Deadlock xảy ra khi:
  - Các giao tác chờ khóa của nhau
  - Không giao tác nào tiếp tục được
- ➔Hệ thống bị tắc nghẽn.

Deadlock (Khóa chết)

- Ví dụ: Cho 2 giao tác T1 và T2
  - T1 khóa Y
  - T2 khóa X
  - T1 muốn khóa X nhưng T2 đang giữ khóa
  - T2 muốn khóa Y nhưng T1 đang giữ khóa
  - T1, T2 chờ nhau ➔ **deadlock** xảy ra

T_1	T_2
Read_Lock(Y)	
Read(Y)	
	Read_Lock(X)
	Read(X)
Write_Lock(X)	
	Write_Lock(Y)

Deadlock (Khóa chết)

- Giải pháp:
  - Xin tất cả khóa cùng lúc
  - Cấp khóa theo thứ tự
  - Phát hiện và phục hồi

Giao thức khóa cơ bản

- Để giải quyết các vấn đề trong điều khiển tương tranh, hệ quản trị CSDL sử dụng nhiều giao thức khóa.
- Một số giao thức khóa cơ bản
  - Khóa nhị phân: đơn giản nhưng ít dùng trong thực tế
  - Khóa đọc/ghi: được sử dụng phổ biến trong hệ quản trị CSDL

Khóa nhị phân

- Có 2 trạng thái:
  - 0: Không bị khóa
  - 1: Đang bị khóa
- Kí hiệu: LOCK(X)
  - X: đơn vị dữ liệu
  - LOCK(X)=1 → X đang bị khóa
  - LOCK(X)=0 → X có thể truy cập

Khóa nhị phân

- 2 thao tác cơ bản:
  - Cấp khóa: LOCK(X)

    → Xin quyền truy cập dữ liệu
  - Giải phòng khóa: UNLOCK(X)

    → Trả lại quyền truy cập dữ liệu

Khóa nhị phân

- Quy trình cấp khóa và giải phòng khóa

Lock(X):

***B**: If LOCK(X)=0 Then //Đơn vị dữ liệu X không bị khóa*

LOCK(X)←1 //thiết lập khóa dữ liệu X

Else

Begin

Wait (until LOCK(X)=0)

Bộ quản lý cấp khóa sẽ cấp khóa cho giao tác

*Go to **B***

End;

Khóa nhị phân

- Quy trình cấp khóa và giải phòng khóa

UnLock(X):

LOCK(X) ← 0; //Giải phóng khóa đơn vị dữ liệu X

If Có giao tác đang đợi cấp khóa đơn vị dữ liệu X thì Bộ quản lý cấp khóa sẽ trao khóa cho giao tác đó

Khóa nhị phân

- Quy tắc của giao tác khóa nhị phân:
  - Giao tác phải Lock(X) trước khi đọc hoặc ghi X
  - Sau khi hoàn thành Unlock(X)
  - Không thể Lock(X) nếu X đang bị khóa bởi giao tác khác
  - Không thể Unlock(X) nếu giao tác không phải là chủ sở hữu khóa

Khóa nhị phân

- Lưu ý:
  - Một giao tác đúng đắn: chỉ được đọc/ghi sau khi xin khóa thành công và trước khi giải phóng khóa
  - Lịch giao tác hợp lệ: Khi một giao tác T đang giữ khóa trên đơn vị dữ liệu X thì giao tác khác không được cấp khóa trên X

Khóa nhị phân

- Ví dụ 1: Cho lịch giao tác S_1 gồm 2 giao tác T_1 và T_2 được biểu diễn như hình bên.
  - Giao tác T_1 và T_2 có đúng đắn không?
  - Lịch giao tác S_1 sử dụng giao thức khóa nhị phân có hợp lệ không?

T_1	T_2
Lock(X) Read(X,t);	
t:=t+50;	
Write(X,t); UnLock(X)	
	Lock(X) Read(X,s);
	s:=s*2;
	Write(X,s); UnLock(X) Lock(Y)
	Read(Y,s);
	s:=s*2;
	Write(Y,s); UnLock(Y)
Lock(Y) Read(Y,t);	
t:=t+50;	
Write(Y,t) UnLock(Y)	

Khóa nhị phân

- Ví dụ 2: Cho lịch giao tác S_2 gồm 3 giao tác T_1, T_2 và T_3 được biểu diễn như hình bên.
  - Các giao tác T_1, T_2 và T_3 có đúng đắn không?
  - Lịch giao tác S_2 sử dụng giao thức khóa nhị phân hợp lệ không?

T_1	T_2	T_3
Lock(X) Lock(Y)		
Read(X)		
Write(Y)		
	Lock(Y)	
UnLock(X)		
UnLock(Y)		
	Read(Y) Write(Y) UnLock(Y)	
		Lock(Y)
		Read(Y)
		UnLock(Y)

Khóa nhị phân

- Ví dụ 3: Cho lịch giao tác S_3 gồm 3 giao tác T_1, T_2 và T_3 được biểu diễn như hình bên.
  - Các giao tác T_1, T_2 và T_3 có đúng đắn không?
  - Lịch giao tác S_3 sử dụng giao thức khóa nhị phân hợp lệ không?

T_1	T_2	T_3
Lock(X)		
Read(X)		
Write(Y)		
UnLock(X)		
UnLock(Y)		
	Lock(Y)	
	Read(Y)	
	Write(Y)	
		Lock(Y)
		Read(Y)
		UnLock(Y)